UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Zanesen / 2327240066
Kelas	SI-5C
Judul project / topik API	Toko Online - Produk API
Tautan repositori	https://github.com/Zanesennn/tugas1-restfull-2327240066
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	21-10-2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

Jelaskan dalam dua sampai tiga kalimat: topik API Anda, entitas utamanya, dan teknologi yang dipakai (misalnya Express.js dengan struktur Route-Controller-Model).

API yang saya buat adalah RESTful API Toko Online untuk mengelola data produk. API ini menyediakan fungsi melihat seluruh produk, melihat satu produk berdasarkan ID, menambahkan, mengubah, menghapus, dan memfilter produk berdasarkan kategori menggunakan Express.js, array di memori, serta pengujian otomatis melalui Postman.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/	Tidak	200	Menampilkan informasi API, identitas mahasiswa, dan daftar endpoint.
2	GET	/products	Tidak	200	Mengambil seluruh data produk; response berupa array langsung.
3	GET	/products/:id	Tidak	200 / 404	Mengambil satu produk berdasarkan ID; 404 jika produk tidak ditemukan.
4	POST	/products	Tidak	201 / 400	Menambahkan produk baru; memvalidasi nama, kategori, dan harga.
5	PUT	/products/:id	Tidak	200 / 400 / 404	Mengubah seluruh field produk berdasarkan ID dan memvalidasi field wajib.
6	DELETE	/products/:id	Tidak	200 / 404	Menghapus produk berdasarkan ID; response sukses memakai data null.
7	GET	/products?kategori=elektronik	Tidak	200	Memfilter produk berdasarkan kategori melalui query string.

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

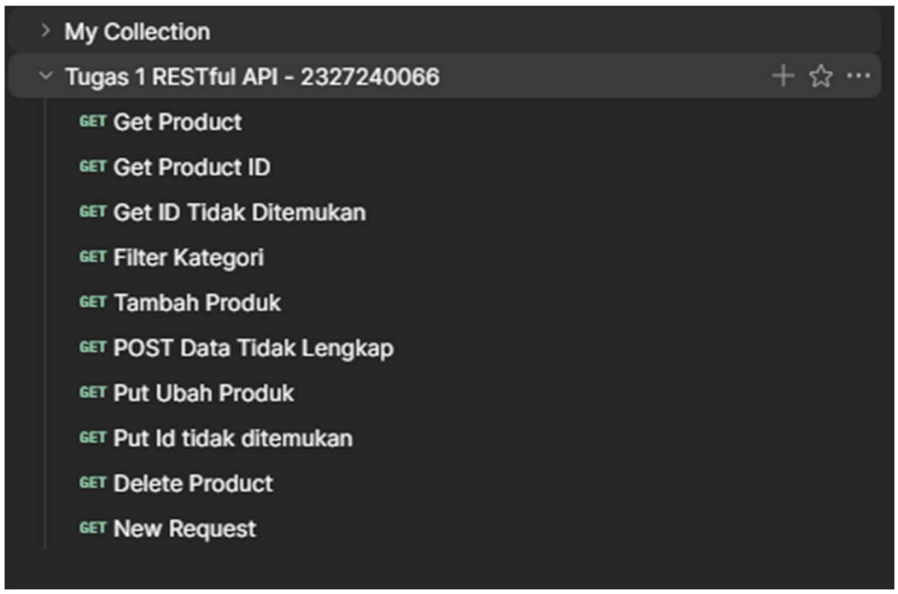
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server lokal Express.js.
2	productsId	Disimpan otomatis oleh script POST	ID produk baru untuk request GET, PUT, dan DELETE.
3	categoryFilter	elektronik	Nilai kategori yang dipakai pada filter GET /products.

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Jelaskan cara Anda mengelompokkan request (misalnya folder CRUD dan folder skenario gagal) dan mengapa urutannya seperti itu.

Request disatukan dalam Collection bernama Toko Online API - Zanesen. GET Info API dan GET Semua Produk diletakkan di awal, lalu POST Tambah Produk dijalankan sebelum GET Produk by ID, PUT, dan DELETE karena script POST menyimpan ID baru ke Environment. Request filter kategori dan skenario gagal (field wajib kosong atau ID tidak ditemukan) digunakan untuk memeriksa respons API pada kondisi normal maupun tidak valid.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Info API	GET	{{base_url}}/	TUGAS 2
2	GET Semua Produk	GET	{{base_url}}/products	TUGAS 2
3	POST Tambah Produk	POST	{{base_url}}/products	TUGAS 2
4	GET Produk by ID	GET	{{base_url}}/products/{{productsId}}	TUGAS 2

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
5	GET Filter Kategori	GET	{{base_url}}/products?kategori={{categoryFilter}}	TUGAS 2
6	POST Validasi Field Kosong	POST	{{base_url}}/products	TUGAS 2
7	GET Produk ID Tidak Ditemukan	GET	{{base_url}}/products/999999	TUGAS 2
8	PUT Ubah Produk	PUT	{{base_url}}/products/{{productsId}}	TUGAS 2
9	PUT Produk ID Tidak Ditemukan	PUT	{{base_url}}/products/999999	TUGAS 2
10	DELETE Produk	DELETE	{{base_url}}/products/{{productsId}}	TUGAS 2
11	DELETE Produk ID Tidak Ditemukan	DELETE	{{base_url}}/products/999999	TUGAS 2

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Info API	200	Objek: nama_mahasiswa=Zanesen, nim=2327240066, topik=2, resource=products, endpoints=array.
2	GET Semua Produk	200	Array langsung; setiap produk memiliki id, nama, kategori, dan harga.
3	POST Tambah Produk	201	status='success'; data.id ada dan tersimpan ke productsId.
4	GET Produk by ID	200	Objek langsung; id sama dengan productsId.
5	GET Filter Kategori	200	Array; kategori tiap produk sama dengan categoryFilter.
6	POST Validasi Field Kosong	400	status='error'; message validasi muncul; data=null.
7	GET Produk ID Tidak Ditemukan	404	status='error'; data=null.
8	PUT Ubah Produk	200	status='success'; data.nama='Headset Gaming Pro'.
9	PUT Produk ID Tidak Ditemukan	404	status='error'; data=null.
10	DELETE Produk	200	status='success'; message menyebut ID; data=null.
11	DELETE Produk ID Tidak Ditemukan	404	status='error'; data=null.

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: GET Semua Produk
pm.test("Status code adalah 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berupa array dan field produk tersedia", function () {
    const body = pm.response.json();
    pm.expect(body).to.be.an("array");
    body.forEach((produk) => {
        pm.expect(produk).to.have.property("id");
        pm.expect(produk).to.have.property("nama");
        pm.expect(produk).to.have.property("kategori");
        pm.expect(produk).to.have.property("harga");
    });
});
```

Penjelasan:

Jelaskan fungsi setiap `pm.test`: apa yang diperiksa dan mengapa pemeriksaan itu penting.

Script GET Semua Produk memeriksa bahwa server memberi status HTTP 200. Test berikutnya memeriksa bahwa body response berbentuk array langsung, sesuai implementasi GET /products, lalu mengecek keberadaan field utama setiap produk. Pemeriksaan ini membantu menemukan perubahan struktur response atau field yang hilang.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: POST Tambah Produk
pm.test("Status code adalah 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Produk berhasil dibuat dan ID tersimpan", function () {
    const body = pm.response.json();
    pm.expect(body.status).to.eql("success");
    pm.expect(body.data).to.be.an("object");
    pm.expect(body.data.id).to.exist;
    pm.expect(body.data.nama).to.eql("Headset Gaming");
    pm.environment.set("productsId", String(body.data.id));
});
```

Penjelasan:

Jelaskan bagaimana data dari respons disimpan, request mana yang memakainya, dan apa yang terjadi bila urutan request ditukar.

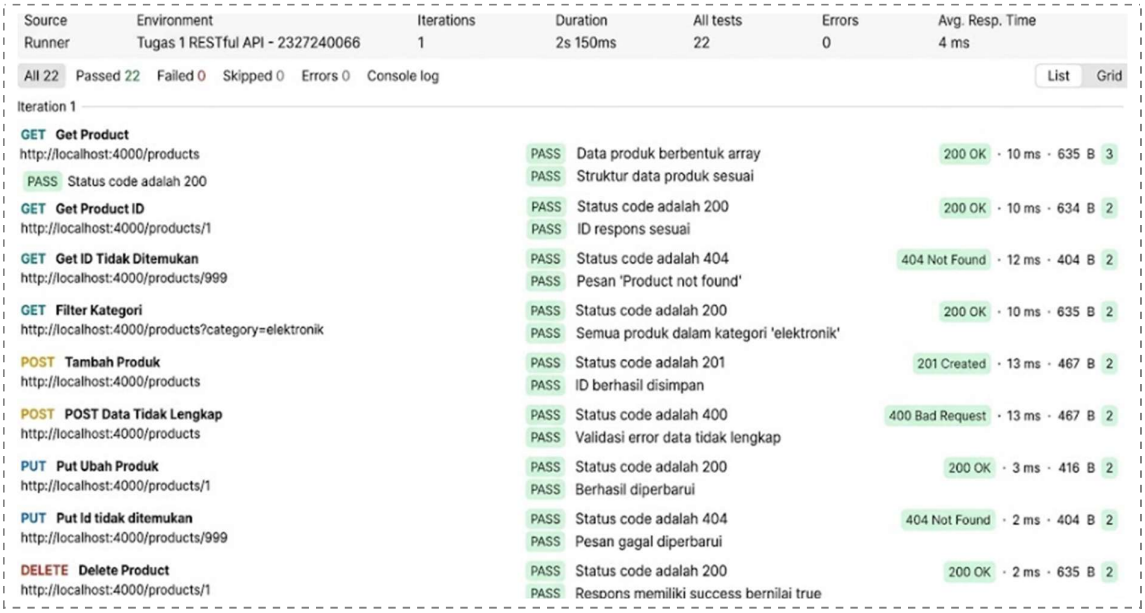
Request POST Tambah Produk mengirim JSON berisi nama, kategori, harga, stok, dan deskripsi. Script memastikan status 201, memeriksa status response success, lalu mengambil data.id dan menyimpannya ke Environment sebagai productsId. Variabel ini dipakai oleh GET Produk by ID, PUT Ubah Produk, dan DELETE Produk; karena itu POST harus dijalankan lebih dulu daripada request yang bergantung pada ID tersebut.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	Tugas 1 RESTful API - 2327240066	1	2s 150ms	22	0	4 ms

All 22 Passed 22 Failed 0 Skipped 0 Errors 0 Console log — List Grid

Iteration 1

GET Get Product http://localhost:4000/products	PASS Data produk berbentuk array PASS Struktur data produk sesuai	200 OK · 10 ms · 635 B	3
PASS Status code adalah 200			
GET Get Product ID http://localhost:4000/products/1	PASS Status code adalah 200 PASS ID respons sesuai	200 OK · 10 ms · 634 B	2
GET Get ID Tidak Ditemukan http://localhost:4000/products/999	PASS Status code adalah 404 PASS Pesan 'Product not found'	404 Not Found · 12 ms · 404 B	2
GET Filter Kategori http://localhost:4000/products?category=elektronik	PASS Status code adalah 200 PASS Semua produk dalam kategori 'elektronik'	200 OK · 10 ms · 635 B	2
POST Tambah Produk http://localhost:4000/products	PASS Status code adalah 201 PASS ID berhasil disimpan	201 Created · 13 ms · 467 B	2
POST POST Data Tidak Lengkap http://localhost:4000/products	PASS Status code adalah 400 PASS Validasi error data tidak lengkap	400 Bad Request · 13 ms · 467 B	2
PUT Put Ubah Produk http://localhost:4000/products/1	PASS Status code adalah 200 PASS Berhasil diperbarui	200 OK · 3 ms · 416 B	2
PUT Put Id tidak ditemukan http://localhost:4000/products/999	PASS Status code adalah 404 PASS Pesan gagal diperbarui	404 Not Found · 2 ms · 404 B	2
DELETE Delete Product http://localhost:4000/products/1	PASS Status code adalah 200 PASS Respons memiliki success bernilai true	200 OK · 2 ms · 635 B	2

Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Jelaskan hasilnya: berapa request dan test yang dijalankan, apakah semuanya lulus, dan apa yang Anda lakukan sebelum menjalankan Runner (misalnya menyalakan ulang server).

Jalankan Collection Toko Online API - Zanesen menggunakan Environment Toko Online - Local setelah server aktif. Collection ini dirancang berisi 11 request dengan dua pm.test pada setiap request, sehingga total rancangan adalah 22 test. Masukkan jumlah test lulus dan gagal berdasarkan hasil Runner yang benar-benar muncul; jangan isi angka lulus sebelum pengujian dilakukan.

4.2 Rekap angka hasil

Nama Collection	Toko Online API - Zanesen
Environment yang dipakai	Toko Online - Local
Jumlah request dijalankan	22
Jumlah test lulus	22
Jumlah test gagal	0
Tanggal dan jam pengujian	21/10/2026

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Diisi berdasarkan hasil aktual	Catat pesan kegagalan yang benar-benar muncul. Jika tidak ada, tulis "Tidak ada kegagalan".	Tuliskan penyebab berdasarkan response atau log yang ditemukan.	Tuliskan perbaikan yang benar-benar dilakukan; jika tidak ada kegagalan, isi "-".

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot Newman jika dikerjakan; jika tidak, tulis: Tidak dikerjakan (opsional).]

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/Zanesennn/tugas1-restfull-2327240066
Lokasi berkas Collection (.json)	
Lokasi berkas Environment (.json)	
Berkas Environment tidak memuat kunci rahasia sungguhan	Belum diperiksa; verifikasi setelah Environment diekspor.
Repositori dapat dibuka dosen	✓ Ya (repository publik)

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Tulis jawaban Anda di sini.

Menurut saya, assertion status code dan pemeriksaan struktur JSON paling berguna. GET /products mengembalikan array langsung, sedangkan POST, PUT, dan DELETE memakai struktur status, message, dan data. Pemeriksaan ID dari hasil POST juga memastikan GET, PUT, dan DELETE memakai produk yang benar.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Tulis jawaban Anda di sini.

Pengujian manual membuat saya memeriksa setiap request satu per satu, sedangkan Collection Runner menjalankan request dan assertion secara berurutan. Bagian yang perlu diperhatikan adalah menyimpan productsId dari response POST dan mengatur urutan request agar ID tersebut tersedia sebelum dipakai oleh GET, PUT, dan DELETE.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	☐
2	Setiap endpoint API saya ada di Collection	☐
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	☐
4	Setiap request punya assertion status code dan struktur JSON	☐
5	Nama setiap test berupa kalimat yang menyatakan harapan	☐
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	☐
7	Server dinyalakan ulang sebelum Runner dijalankan	☐
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	☐
9	Berkas Environment tidak memuat kunci rahasia sungguhan	☐
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	☐

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen